

I. System V^[1]

I-a) (Global) Function Calling Sequence

1) Register

1. %rbp, %rbx and %r12-%r15 belong to the calling function and the called function is required to preserve their values in its local stack frame
2. x87 control word and control bits of MXCSR are callee-saved (preserved across calls); otherwise, x87 status word and status bits are caller-saved
3. "x87 mode upon entry to function" requires to issue (f)emms after using MMX registers, same to %rFLAGS's direction flag DF must be clear on function entry and return; other user flags are not callee-saved

2) Stack Frame

Position	Contents	Frame
8n+16 (%rbp)	memory argument eightbyte <i>n</i>	Previous
	...	
16 (%rbp)	memory argument eightbyte 0	Current
8 (%rbp)	return address	
0 (%rbp)	previous %rbp value	
-8 (%rbp)	unspecified	
	...	
0 (%rsp)	variable size	
-128 (%rsp)	red zone	

1. Each function has a frame on the run-time stack growing downwards from high addresses. The stack needs to be 16 (32 or 64, if __m256 or __m512 is passed on stack) byte aligned immediately before the call instruction is executed
2. 128-byte red zone is reserved and shall not be modified by signal or interrupt handlers; leaf functions may use this area for their entire stack frame, rather than adjusting the stack pointer in the prologue and epilogue
3. The size of each argument gets rounded up to 8 bytes

3) Parameter Class

1. **INTEGER** → Register
{rule} integral types that fit into one of the **general purpose registers**

{sample} _Bool, char, short, int, long, long long, and pointers, _BigInt($N \leq 64$)

2. **SSE** → Register

{rule} types that fit into a vector register

{sample} _Float16, float, double, _Decimal32, _Decimal64 and __m64

3. **SSEUP** → Register

{rule} types that occupy the upper parts of a vector register, the least significant 8Byte chunk belongs to SSE

{sample} __float128, _Decimal128, __m128, __m256, __m512

4. **X87, X87UP**

{rule} types that will be returned via the x87 FPU {sample} 64-bit mantissa of **long double** belongs to class X87, 16-bit exponent plus 6 bytes of padding belongs to class X87UP

5. **COMPLEX_X87**

{rule} types that will be returned via the x87 FPU

{sample} complex long double

6. **NO_CLASS**

{rule} used for padding and empty structures and unions

7. **MEMORY**

{rule} types that will be passed and returned in memory via the stack

Aggregate (structures and arrays) and union classification

1. size is larger than eight 8Byte or unaligned fields → **MEMORY**
2. non-trivial object (can't be passed by value, so invisible reference) → **INTEGER**
3. aggregate exceeds a 8Byte, each 8Byte is classified separately → **NO_CLASS** initialization
4. For each 8Byte, recursively classify the (sub)fields that occupy it, then merge their classes to determine that 8Byte's final class:

{rule}

1. Same classes $X \rightarrow X$
2. $\exists$ **NO_CLASS** → other class
3. $\exists$ **MEMORY** → **MEMORY**
4. $\exists$ **INTEGER** → **INTEGER**
5. $\exists$ **X87 or X87UP or COMPLEX_X87** → **MEMORY**
6. Otherwise, **SSE**

5. Post merger cleanup

{rule}

1. The whole argument is passed in memory if $\exists$ **MEMORY** or **X87UP** is not preceded by **X87** or if aggregate exceeds two 8Byte and first 8Byte isn't **SSE** or any other 8Byte isn't **SSEUP**
2. If **SSEUP** is not preceded by **SSE** or **SSEUP**, it is converted to **SSE**

NOTE

1. `__int128` can be treated as `struct { long, long }` with 16-byte boundary alignment exception
2. complex `T` ($T \in \{ _Float16, \text{float}, \text{double} \text{ or } __float128 \}$) can be treated as `struct { T, T }`

4) Parameter Passing

1. Registers get assigned in left-to-right order. Once registers are assigned, the arguments passed in memory are pushed on the stack in reversed (right-to-left) order
2. `%r11` is deliberately left unused for argument passing and preservation so that PLT code need not spill (temporarily save to memory) any registers when computing the address to which control needs to be transferred
3. If there are no registers available for any 8Byte of an argument, the whole argument is passed on the stack. If registers have already been assigned for some 8Bytes of such an argument, the assignments get reverted

5) Parameter Returning

For class MEMORY, caller provides space for the return value and passes the address of this storage in `%rdi` (`== %rax` on return). In effect, this address becomes a “hidden” first argument. This storage mustn't overlap any data visible to callee through other names than this argument

<pre>typedef struct { int a, b; double d; } structparm; structparm s; int e, f, g, h, i, j, k; long double ld; double m, n; __m256 y; __m512 z; extern void func (int e, int f, structparm s, int g, int h, long double ld, double m, __m256 y, __m512 z, double n, int i, int j, int k); func (e, f, s, g, h, ld, m, y, z, n, i, j, k);</pre>																									
<table><tr><th>General Purpose Registers</th><th>Floating Point Registers</th><th>Stack Frame Offset</th></tr><tr><td>%rdi: e</td><td>%xmm0: s.d</td><td>0: ld</td></tr><tr><td>%rsi: f</td><td>%xmm1: m</td><td>16: j</td></tr><tr><td>%rdx: s.a,s.b</td><td>%ymm2: y</td><td>24: k</td></tr><tr><td>%rcx: g</td><td>%xmm3: z</td><td></td></tr><tr><td>%r8: h</td><td>%xmm4: n</td><td></td></tr><tr><td>%r9: i</td><td></td><td></td></tr></table>			General Purpose Registers	Floating Point Registers	Stack Frame Offset	%rdi: e	%xmm0: s.d	0: ld	%rsi: f	%xmm1: m	16: j	%rdx: s.a,s.b	%ymm2: y	24: k	%rcx: g	%xmm3: z		%r8: h	%xmm4: n		%r9: i				
General Purpose Registers	Floating Point Registers	Stack Frame Offset																							
%rdi: e	%xmm0: s.d	0: ld																							
%rsi: f	%xmm1: m	16: j																							
%rdx: s.a,s.b	%ymm2: y	24: k																							
%rcx: g	%xmm3: z																								
%r8: h	%xmm4: n																								
%r9: i																									

I-b) Operating System Interface

1) Exception Interface

1. **Type** synchronous, floating-point/coprocessor or asynchronous in which first two exception types, being caused by instruction execution, can be explicitly generated by a process
2. **Classification** faults, traps, and aborts

Number	Exception name	Signal	Code	Reason
0	divide error fault	SIGFPE	FPE_FLTDIV	floating-point divide by zero
1	single step trap/fault	SIGTRAP	FPE_FLTOVF	floating-point overflow
2	non-maskable interrupt	none	FPE_FLTUND	floating-point underflow
3	breakpoint trap	SIGTRAP	FPE_FLTRES	floating-point inexact result
4	overflow trap	SIGSEGV	FPE_FLTINV	invalid floating-point operation
5	(reserved)			
6	invalid opcode fault	SIGILL		
7	no coprocessor fault	SIGFPE		
8	double fault abort	none		
9	coprocessor overrun abort	SIGSEGV		
10	invalid TSS fault	none		
11	segment not present fault	none		
12	stack exception fault	SIGSEGV		
13	general protection fault/abort	SIGSEGV		
14	page fault	SIGSEGV		
15	(reserved)			
16	coprocessor error fault	SIGFPE		
other	(unspecified)	SIGILL		

2) Virtual Address Space

1. Only required to handle 48-bit addresses, therefore, conforming processes may only use addresses up to 0x7fffffff
2. Systems are permitted to use any power-of-two page size in [4KB, 64KB]
3. Conceptually, processes has the full address space, but in practice process size is limited by system-reserved space, per-process limits, and available combined physical memory and secondary storage
4. Dereferencing null pointer is erroneous, process shouldn't expect 0x0 to be a valid address
5. Applications may control their memory assignments

Virtual Address Configuration		Conventional Segment Arrangements	
0xffffffffffffffff	Reserved system area	0x800000000000	...
	...		Dynamic segments
0x800000000000	Dynamic segments		Stack segment
	...		...
0	Process segments		...
		0x400000	Data segments
		0	...
			Text segments
			Unmapped

I-c) Process Initialization

1) Special Registers

x87 Control Word			MXCSR Status Bits			rFLAGS Bits		
Field	Value	Note	Field	Value	Note	Field	Value	Note
RC	0	Round to nearest	FZ	0	Do not flush to zero	DF	0	Direction forward
PC	11	Double extended precision	RC	0	Round to nearest	CF	0	No carry
PM	1	Precision masked	PM	1	Precision masked	PF	0	Even parity
UM	1	Underflow masked	UM	1	Underflow masked	AF	0	No auxiliary carry
OM	1	Overflow masked	OM	1	Overflow masked	ZF	0	No zero result
ZM	1	Zero divide masked	ZM	1	Zero divide masked	SF	0	Unsigned result
DM	1	De-normal operand masked	DM	1	De-normal operand masked	OF	0	No overflow occurred
IM	1	Invalid operation masked	IM	1	Invalid operation masked			
			DAZ	0	De-normals are not zero			

2) Stack

1. exec (BA_OS) creates the machine state for a new process. Language-specific startup code transforms it into the state required by the language. For C, _start prepares argc, argv, and envp, then calls main()
2. When main() returns, its value is passed to exit(); if exit() is overridden and returns, _exit(), which must be immune to user interposition, terminates the process
3. When _start is called, %rbp ← 0, %rdx ← a function pointer that the application should register with atexit (BA_OS) and initial process stack:

Purpose	Start Address	Length
Unspecified	High Addresses	
Information block, including argument strings, environment strings, auxiliary information ...		varies
Unspecified		
Null auxiliary vector entry		1 eightbyte
Auxiliary vector entries ...		2 eightbytes each
0		eightbyte
Environment pointers ...		1 eightbyte each
0	8+8*argc+%rsp	eightbyte
Argument pointers	8+%rsp	argc eightbytes
Argument count	%rsp	eightbyte
Undefined	Low Addresses	

New thread inherits floating-point state of parent thread and state is private to thread thereafter

3) Auxiliary Vector

1. At process creation the system may pass control to an interpreter program. When this happens, the system places either an entry of type AT_EXECFD or one of type AT_PHDR in the auxiliary vector
2. The system may create the memory image of the application program before passing control to the interpreter program
3. Interpreter program eventually transfer control to entry point specified in AT_ENTRY entry

I-d) Program Loading

1. mapping file segments, where $p_{vaddr} \equiv p_{offset} \pmod{pg_{size}}$ for efficient mapping, to virtual memory segments
2. last file page of text **segment** may contain first page of data segment and last data page may contain file information not relevant to the running process to save space
3. segments' addresses are adjusted to ensure each logical page in the address space has a single set of permissions as if each segment were complete and separate
4. end of the data segment requires special handling, zero out, for uninitialized data; thus, (irrelevant) information not in the logical memory page must be zeroed out
5. executable file typically contains absolute code and shared object typically contains PIC

I-e) Dynamic Linking

1) Dynamic Section

1. Dynamic section entries give information, some is processor-specific, including the interpretation of some entries in the dynamic structure, to the dynamic linker
2. AMD64 Dynamic Array Tags (DT_X86_64_PLT, DT_X86_64_PLTSZ, DT_X86_64_PLTENT)
 - d_tag - together with r_offset and r_addend fields of R_X86-64_JUMP_SLOT enable dynamic linker rewrite indirect jumps in PLT entries into direct branches by retrieving target address from GOT, provided PLT entries follow a strict, standardized format:
 1. **Uniform Layout** every PLT entry must have the same size, proper alignment, and a single entry point - only enter (jump to) - at byte 0
 2. **Safe Branch Structure** entries must begin with an indirect branch over the GOT entry, optionally preceded by ENDBR64 for (Intel) CET, that can be safely swapped to a direct branch without side effects.

2) Program Interpreter

1. **Path** /lib/ld64.so.1 and /lib/ldx32.so.1
2. **Linux Path** /lib64/ld-linux-x86-64.so.2 and /libx32/ld-linux-x32.so.2

3) Symbolic Reference Resolving

```
.PLT0: pushq    GOT+8(%rip)      # GOT[1] - identifying information
      jmp     *GOT+16(%rip)    # GOT[2] - dynamic linker's resolver address
      nopl    0x0(%rax)
.PLT1: jmp     *name1@GOTPCREL(%rip) # 16 bytes from .PLT0
      pushq   $index1
      jmp     .PLT0
.PLT2: ...                    # 16 bytes from .PLT1
```

1. Step

1. Each shared object file in the process image has its own PLT, and control transfers to a PLT entry only from within the same object file
2. Initially GOT holds address of following pushq instruction, not real address of name1
3. pushq pushes a relocation index on stack, indexing into the relocation table defined by DT_JMPREL dynamic section entry
4. The designated relocation entry will have type R_X86_64_JUMP_SLOT, its offset will specify GOT entry used in previous jmp instruction, and it contains a symbol table index for appropriate symbol - name1
5. When dynamic linker receives control, after jmp GOT[2], it unwinds the stack, looks at designated relocation entry, finds symbol's value, stores name1 actual address in GOT

2. Note

1. LB_BIND_NOW environment variable can change dynamic linking behavior that evaluates PLT entries before transferring control to program
2. **Have not yet understood** R_X86_64_TLSDESC relocations support lazy relocation using dedicated PLT, GOT entries given by DT_TLSDESC_PLT, DT_TLSDESC_GOT

4) Initialization and Termination

1. Initialization functions are specified by DT_INIT, DT_INIT_ARRAY and DT_PREINIT_ARRAY
2. Termination functions are specified by DT_FINI and DT_FINI_ARRAY

[1]: [System V ABI](#)